

CMPT 276 - Assignment 4

Matthew Pham and Ryan Fung

For assignment 4, we selected a portion of the code we had jointly implemented and systematically examined it for code smells. Specifically, we refactored the classes `MainCharacter.java` and `ConcordOfficer.java` as these two classes had a lot to clean up to reduce complexity, improve maintainability, and improve testability. Through this process, we identified and addressed at least **8** distinct code smells. Each refactoring was followed by regression testing to ensure that our game behaviour remained consistent. This report documents the identified code smells, the reasoning behind each refactoring, and references to relevant commits for traceability.

In `MainCharacter.java`, several methods were **too long**, which made the class **too large**, and could be improved through refactoring. For instance, `eventMoveUp()`, `eventMoveDown()`, `eventMoveLeft()`, and `eventMoveRight()` each handled similar behaviour, differing only in direction, which caused **code duplication**. To fix this, we introduced a new method, `changeImageDirection()`, which centralizes the directional logic based on key inputs (bd47d66). Another instance of a long method that required refactoring was the collision detection between the Concord Officer and the Main Character, Cone, and/or the Parked Car. To address this, we introduced a new method called `officerCollision()` to encapsulate this logic. This made the code more readable and easier to test (115dd20).

We identified **low cohesion** in methods like `move()`, `eventMove()`, and `delayedMove()`. To address this, we applied the Move Method refactoring technique by relocating `move()` and `eventMove()` into `delayedMove()`, thereby improving cohesion and clarity (8a39274).

Collision detection logic, such as checking the player's interaction with cones or parked cars, was initially **poorly structured** and exhibited mild **feature envy**. Specifically, `delayedMove()` directly accessed and manipulated the Board object's attributes to handle gameplay consequences of collisions. We refactored this by creating a dedicated `checkCollision()` method, which centralized collision handling and made the code more cohesive and easier to read (eaf0344). Another example of poorly structured code was the logic for checking whether the player is in an entrance or exit cell. To improve this, we refactored the logic into a new method called `isEntranceOrExitCell()`, which is now called within `tick()` (3bc4a4b).

In `ConcordOfficer.java`, we noticed that the constructor was **too long** and had **low cohesion**. A large block of logic for setting up the officer's timer was embedded directly, which made the constructor harder to read and maintain. It also affected the reusability and testability of the timer logic. To fix this, we extracted the timer setup logic into a new private method called `startOfficerTimer()`. This improves modularity and aligns the constructor with clean code practices (7b71817).

Immediately after creating the `startOfficerTimer()` method, we noticed that the logic related to collision handling and game-over conditions was embedded within the lambda function of the timer. This made the timer logic bloated and resulted in **high coupling** of movement, collision, and UI logic in a single place. We moved this logic into a new private method called `officerCollision()` to fix this. This improves cohesion and separates responsibilities between methods (115dd20).